

LLM Based Chatbot Assistant

A PROJECT REPORT

Submitted by

Vignesh Kumar Reddy V (CB.SC.I5DAS21166)

*in partial fulfillment of the requirements for the award of the
degree of*

INTEGRATED MASTER OF SCIENCE

IN
DATA SCIENCE



Department of Mathematics

AMRITA SCHOOL OF PHYSICAL SCIENCES

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE 641112

APRIL 2025

DEPARTMENT OF MATHEMATICS
AMRITA SCHOOL OF PHYSICAL SCIENCES,
Coimbatore - 641112



BONAFIDE CERTIFICATE

This is to certify that the project report entitled **LLM Based Chatbot Assistant** submitted by Vignesh Kumar Reddy V, **Roll no: CB.SC.I5DAS21166** in partial fulfillment of the requirements for the award of the **Degree of Integrated Master of Science in Data Science** is a bonafide record of the work carried out under my guidance and supervision at Amrita School of Physical Sciences, Coimbatore.

Project Coordinator

Project Advisor

Chairperson
Department of Mathematics
Dr. K. Somasundaram

The project was evaluated by us on:

Internal Examiner

Internal Examiner

DEPARTMENT OF MATHEMATICS
AMRITA SCHOOL OF PHYSICAL SCIENCES,
Coimbatore - 641112



DECLARATION

I, Vignesh Kumar Reddy V, **Roll no: CB.SC.I5DAS21166** hereby declare that this project report entitled “**LLM Based Chatbot Assistant**” is the record of the original work done by me at Department of Mathematics, Amrita School of Physical Sciences, Coimbatore. To the best of my knowledge this work has not formed the basis for the award of any degree / diploma/ associateship / fellowship or a similar award to any candidate in any university/institutions.

Date:

Place:

Signature of the Student

COUNTERSIGNED

Dr. J. Mahalakshmi
Project Coordinator,
Department of Mathematics,
School of Physical Sciences,
Amrita Vishwa Vidyapeetham
Coimbatore-641112, Tamil Nadu, India.

Acknowledgements

I would take this opportunity to thank everyone who helped me with this project. I would like to thank my Project Coordinator Dr. J. Mahalakshmi, Assistant Professor, Department of Mathematics, Amrita Vishwa Vidyapeetham, who constantly guided me. I would like to thank him for taking a considerable amount of time from his busy schedule to help with this mini project.

I take this opportunity to thank Dr. Sasangan Ramanathan, Dean Engineering, for providing the opportunity to access these facilities for the well-being of my mini project.

I would like to express my sincere gratitude towards Dr Somasundaram, Chairperson and all the faculty members of the Department of Mathematics, Amrita Vishwa Vidyapeetham for supporting the vision that I believed in which has made this mini project a reality.

Lastly, I am especially indebted to my parents, family members and friends for their support, motivation, love and affection that have providing me with immense motivation and strength to finish this project successfully.

(Vignesh Kumar Reddy V)

Contents

BONAFIDE CERTIFICATE.....	2
Acknowledgements.....	4
ABSTRACT	7
CHAPTER 1 - INTRODUCTION.....	8
Introduction	9
1.1. Background and Motivation	9
1.2. Purpose of the Project	10
1.3. Key Features Overview	10
1.4. Scope of the Project.....	10
CHAPTER 2 - Problem Statement & Objectives	11
2.1. Challenges in Customer Interaction for Small Businesses	12
2.2. Limitations of Traditional Chatbots	12
2.3. Limitations of Traditional Chatbots	12
2.4. Key Objectives.....	13
CHAPTER 3 - System Design & Architecture.....	14
3.1. High-Level System Overview.....	15
3.2. Data Flow Diagram.....	15
3.3. Modules and Their Responsibilities	17
Chapter 4: Technologies & Tools Used.....	18
4.1 FastAPI for Backend	19
4.2 HTML/CSS/JavaScript (AJAX) for Frontend.....	19
4.3 Together AI API for LLaMA 70B.....	19
4.4 FAISS with Cosine Similarity	20
4.5 FP-Growth for Recommendation.....	20
4.6 MongoDB and FastAPI Session Storage.....	20

Chapter 5: Implementation.....	21
5.1. Real-Time Communication and Session Lifecycle Management	22
5.2. Intelligent Agent Coordination and Modular Processing Pipeline	22
Chapter 6: Results, Challenges & Future Work.....	24
6.1. System Outcomes, Performance Insights, and Development Challenges	25
6.2 Scope for Improvement and Future Enhancements.....	26
 Conclusion.....	 27
Conclusion.....	28
Outputs.....	29
References	31
References	32

ABSTRACT

The objective of this project is to build an AI-powered chatbot system that combines several components to provide users with more efficient assistance. The goal is to develop a system that can comprehend user needs, offer recommendations, retrieve pertinent data, and reply in a meaningful manner—going beyond a simple question-answer bot. It is primarily intended for use in coffee shops, where the chatbot can make menu recommendations, respond to frequently asked questions, and recall previous exchanges to offer a more tailored experience.

Plenty of advanced techniques are employed to make the system intelligent. Based on previous user behavior, a recommendation system that can recommend popular item combinations is constructed using the FP-Growth algorithm. FAISS-powered RAG (Retrieval-Augmented Generation) improves the quality of the chatbot's responses by assisting it in retrieving pertinent and accurate content from external knowledge bases or stored documents. The system helps the chatbot remember user preferences and past interactions by storing both structured (like order history) and unstructured (like chat logs) data using MongoDB and FastAPI Session storages.

FastAPI is used in the development of the project's backend, which manages database operations, session tracking, and model communication. Users can communicate with the bot in real time through the frontend's clear and uncomplicated user interface. The backend and frontend are separated to guarantee seamless operation and facilitate the management of various project components. All things considered, this system combines database integration, agents, retrieval, and recommendation to produce a more sophisticated and practical chatbot for everyday use.

CHAPTER 1 - INTRODUCTION

Chapter 1

Introduction

This project is about building a smart chatbot for a coffee shop that can talk to users, understand what they want, and recommend items based on what they've already asked or bought. It's not just a regular chatbot—it remembers past chats during the session, gives smart replies using a large language model (LLaMA 70B), and suggests things using a recommendation engine. It's like a virtual barista that doesn't just take orders but helps customers make better choices.

I wanted it to feel simple but useful, especially for small shops that may not have big servers or complex systems. That's why I used the Together AI public API to access the LLaMA 70B model instead of trying to run it locally, which would have been too heavy for a normal laptop. Using WebSockets made the conversation feel real-time without reloading the page every time. The backend, built with FastAPI, handles storing the chat context and smoothly connects everything. It's not a flashy app, but it's something real that works—and I learned a lot while building it.

1.1. Background and Motivation

Most small businesses don't have the luxury of hiring full-time staff for handling every customer query. That's where an AI-powered chatbot comes in handy. The motivation behind this project was to make a chatbot that doesn't just reply to questions, but actually remembers the conversation and gives helpful, personalized replies. I wanted something that feels more like a real conversation, not just an automated response.

Also, I wanted to see how far I could push open-source tools and APIs to create something that works smoothly even on simple systems like laptops, without needing huge infrastructure. I was also interested in trying out FAISS and the FP-Growth algorithm together to see how document similarity and recommendations can work side by side. This project gave me a chance to explore those tools in a useful way.

1.2. Purpose of the Project

The purpose of this project is to create a chatbot that not only answers common questions but also helps customers discover new products. Using the LLaMA 70B model for understanding natural language, the chatbot can converse naturally with users. The FP-Growth recommendation system enhances this by suggesting drinks based on past preferences. Together, these technologies aim to improve the overall customer experience and make interactions more personal and relevant.

1.3. Key Features Overview

The bot uses FAISS with cosine similarity to retrieve documents that are most related to the user's message. From those, it understands the context and uses an FP-Growth recommendation agent to give suggestions. It's not just random—it actually recommends based on frequently brought together items.

The messages between the website and backend are sent using WebSockets, so there's no need to reload the page every time. Sessions are managed using FastAPI's storage, and once the session ends, everything is saved to MongoDB—both user messages and bot replies.

1.4. Scope of the Project

This chatbot works well in a coffee shop setting, and could easily be adapted for other small businesses. It handles chatting, recommendations, and document retrieval within a session. Reloading the page doesn't break the chat, which is a nice plus.

However, once the user closes the tab, the session data is lost unless it was saved. Since I'm using the LLaMA 70B model through an API, the project depends on that external service, so it needs internet access and API availability. Also, it doesn't do deep natural language processing—it's more about using similarity and logic to reply smartly.

CHAPTER 2 - Problem Statement & Objectives

Chapter 2

2.1. Challenges in Customer Interaction for Small Businesses

In today's fast-paced world, customers expect quick and helpful responses. For small businesses like independent coffee shops, managing real-time customer interaction can be tough, especially without a large team or advanced infrastructure. Questions like "What's on the menu?", "What do you recommend?", or "Do you have any seasonal drinks?" come up often, and it's not always possible to respond instantly or personally. Manual replies on social media or websites can lead to delays, and repeated queries can become time-consuming. A smarter system that can understand, respond, and even suggest based on user interest could really help improve the customer experience.

2.2. Limitations of Traditional Chatbots

- 1) Most traditional chatbots rely on rule-based logic or keyword matching.
- 2) Rule-based bots can't handle dynamic or multi-turn conversations effectively.
Example: The bot may forget previous context, such as a discussion about coffee types.
- 3) They lack personalization, meaning they cannot suggest items or respond based on past interactions.
- 4) Traditional bots do not learn or evolve over time, which limits their adaptability.
- 5) This leads to a robotic and often frustrating user experience that doesn't meet customer expectations for interactive, personalized service.

2.3. Limitations of Traditional Chatbots

The main aim of this project is to build an intelligent chatbot system tailored for a coffee shop environment, with a focus on natural and useful interactions. Rather than sticking to static responses, the bot should be able to retrieve relevant information from documents, recommend items using data-driven logic, and maintain an ongoing conversation without losing context. The idea is to blend fast performance with meaningful replies — so that customers feel like they're chatting with a helpful virtual assistant, not just a script. The system should also be simple and efficient enough to work on lightweight web infrastructure, making it practical for small business deployment.

2.4. Key Objectives

To achieve this aim, the project is broken down into clear and focused objectives:

- Implement a Retrieval-Augmented Generation (RAG) system using FAISS and cosine similarity. This will help the bot fetch the most relevant document chunks based on what the user says, ensuring more accurate replies.
- Integrate an intelligent recommendation agent that applies the FP-Growth algorithm. This agent will analyze past patterns to suggest drinks or combos that match customer interests, improving engagement.
- Store and manage user sessions on the backend using FastAPI's session capabilities. This enables the bot to remember previous queries within the same session and provide more coherent replies.
- Set up WebSocket-based communication between the frontend and backend. This allows messages to be exchanged in real-time without needing to refresh the webpage, making the chat feel smoother and faster.
- Automatically store past session data — including user queries and bot responses — in MongoDB once the session expires. This creates a simple history system that can be used for analysis or future upgrades.
- Build a lightweight frontend using HTML, CSS, AJAX, and JavaScript. The goal is to keep the interface clean and fast, avoiding full-page reloads and providing a smooth user experience, especially for casual users visiting the site.

CHAPTER 3 - System Design & Architecture

3.1. High-Level System Overview

The intelligent chat system is designed to streamline customer interactions by leveraging a modular architecture that processes user input, identifies intents, and responds through specialized agents. At the core, the system integrates FastAPI for backend processing, MongoDB for persistent storage, and Together AI's LLaMA 70B for natural language understanding and generation.

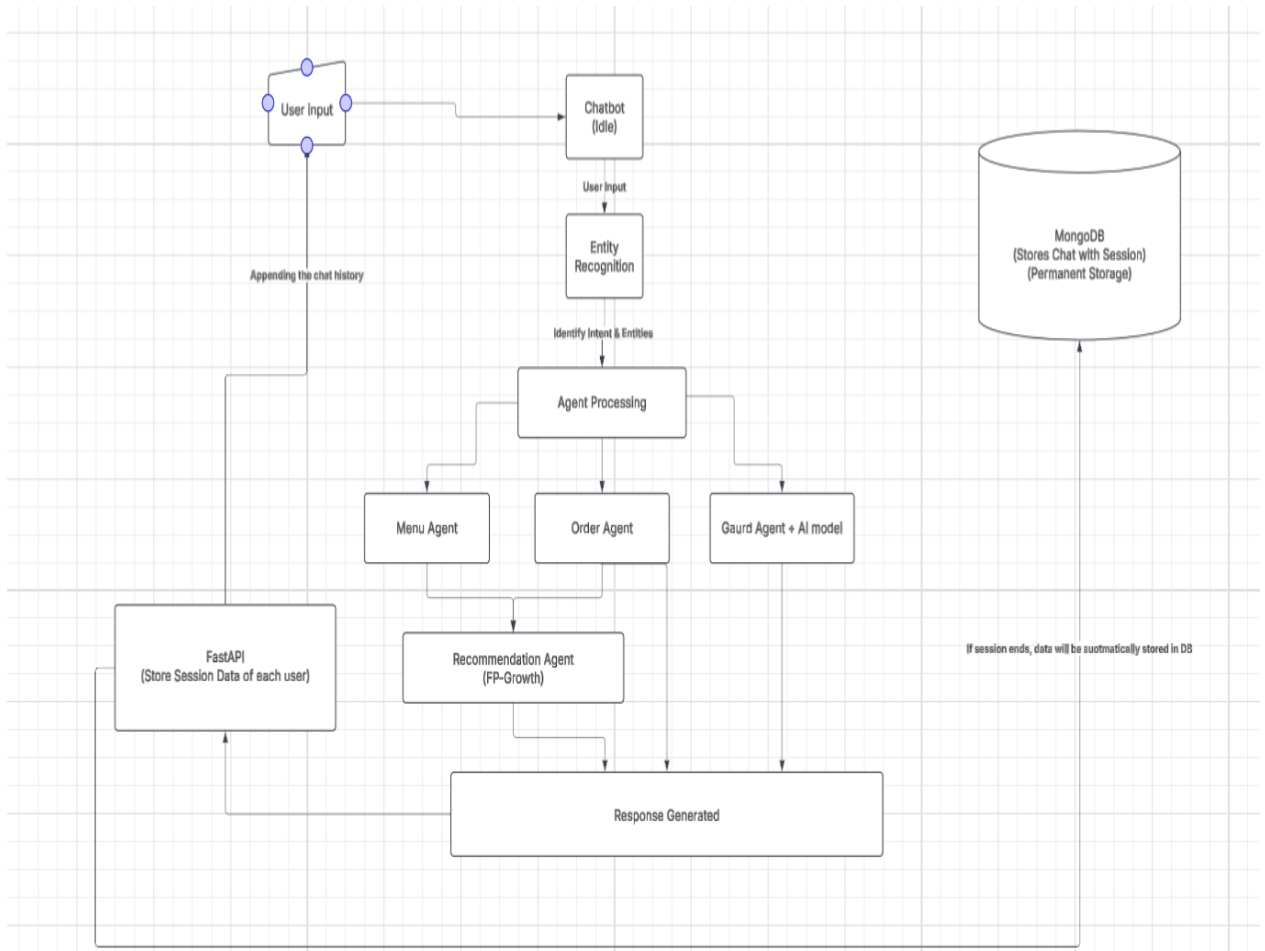
The system ensures session-based interaction, enabling a continuous, contextual, and personalized user experience. This is accomplished through components responsible for intent recognition, recommendation, menu handling, ordering, and security (guard agent), culminating in a coherent response mechanism.

3.2. Data Flow Diagram

The data flow begins with the **User Input**, which is routed to the **Chatbot (Idle)** state. Upon receiving input, the system activates **Intent Recognition** to analyze and extract key entities and intents. This information is passed to the **Agent Processing** layer, where specialized agents—**Menu Agent**, **Order Agent**, **Recommendation Agent**, and **Guard Agent**—handle the task based on the recognized intent.

Each of these agents contributes to the **Response Generation** module, which sends the final response back to the user. Meanwhile, **FastAPI** manages real-time session data, appending chat history and ensuring state continuity.

Additionally, when the session ends or times out, all data is automatically stored in **MongoDB** for permanent archival.



3.3. Modules and Their Responsibilities

Module	Responsibility
main.py	Acts as the central entry point connecting frontend with backend logic via FastAPI and WebSocket communication.
chat_terminal.py	Manages session lifecycle—initiating and terminating sessions, and routing messages to the chatbot.
chatbot.py	Interfaces with the Together AI API (LLaMA 70B), managing intent recognition, response generation, and logic dispatching.
menu_agent.py	Uses FAISS-based RAG (Retrieval-Augmented Generation) to retrieve relevant menu items in response to user queries.
recommendation_agent.py	Analyzes retrieved items and user interactions to recommend frequently bought-together items using the FP-Growth algorithm.
MongoDB	Stores full chat sessions persistently after session termination. Acts as long-term storage.
FastAPI (Session Layer)	Maintains context-aware session data for active users, enabling continuity in user experience.
LLaMA Model (Together AI)	Handles intent recognition, order understanding, and performs basic guard logic (content moderation).

Chapter 4: Technologies & Tools Used

4.1 FastAPI for Backend

1. FastAPI serves as the backbone of the system's backend infrastructure. It is used for:
2. Creating RESTful APIs for data communication between frontend and backend.
3. Managing session state for each user, enabling personalized and context-aware interactions.
4. Facilitating WebSocket connections for real-time message delivery.
5. Its asynchronous capabilities and high performance make it ideal for handling multiple concurrent user sessions.

4.2 HTML/CSS/JavaScript (AJAX) for Frontend

1. The frontend of the chat system is built using:
2. **HTML/CSS** for structuring and styling the user interface.
3. **JavaScript (with AJAX)** for making asynchronous calls to the backend without reloading the page.
4. **WebSocket integration** to maintain a persistent connection for real-time communication with the chatbot.

4.3 Together AI API for LLaMA 70B

The core language model is LLaMA 70B accessed through the Together AI API. It is responsible for:

- Intent recognition and entity extraction from user input.
- Generating coherent and contextually appropriate responses.
- Handling order-related queries and basic guard logic (moderating unsafe content).

The model's powerful language capabilities allow the system to handle complex interactions naturally.

4.4 FAISS with Cosine Similarity

FAISS (Facebook AI Similarity Search) is utilized in the `menu_agent.py` for:

- Performing fast and efficient vector-based similarity searches over menu items.
- Implementing Retrieval-Augmented Generation (RAG) where relevant items are fetched based on embeddings.
- Supporting cosine similarity for accurate relevance matching between queries and menu entries.

4.5 FP-Growth for Recommendation

The FP-Growth algorithm, used in `recommendation_agent.py`, enables:

- Discovery of frequent itemsets from previously retrieved menu selections.
- Providing recommendations based on commonly purchased item combinations.
- Enhancing user experience through personalized suggestions that mimic human-like assistance.

4.6 MongoDB and FastAPI Session Storage

- **MongoDB** is used as the **permanent storage** solution for chat data. When a user session ends, all related conversation logs and session metadata are stored in MongoDB. This ensures long-term accessibility, scalability, and ease of querying historical interactions.
- **FastAPI Session Storage** is employed for **temporary, in-session context management**. It retains user-specific data like message history and session variables throughout the chat duration. This enables context-aware responses and personalized interaction during an active session.

Chapter 5: Implementation

5.1. Real-Time Communication and Session Lifecycle Management

The core functionality of the chatbot revolves around delivering real-time, context-aware communication, which is achieved through FastAPI's WebSocket support. Unlike traditional HTTP-based request-response models, WebSockets allow for persistent, bidirectional communication, making the chatbot feel interactive and responsive, just like a human conversational partner.

The communication begins in the `main.py` file, where the WebSocket route is established. When a user connects from the frontend, a WebSocket session is initiated. This connection is persistent and remains open throughout the chat, allowing for immediate back-and-forth communication between the user and the backend.

Once the connection is established, `chat_terminal.py` becomes the session manager. It is responsible for creating a new user session, tracking its activity, and ensuring it is closed appropriately after completion or timeout. Each message received through the WebSocket is routed through this terminal, which appends it to an in-memory session state managed via FastAPI's session storage. This temporary storage holds conversation context—such as user inputs, responses, and flags for specific intents—allowing the chatbot to maintain awareness over time.

When the session ends, either naturally or due to inactivity, the full session data is serialized and written to MongoDB. MongoDB acts as the system's persistent storage layer, ensuring that no chat history is lost. This stored data not only serves as a log for historical reference but also enables analytics, future personalization, and system improvement based on user behavior. This lifecycle—spanning from WebSocket initiation, through active session handling, to persistent storage—ensures both responsiveness and robustness, making the chatbot scalable and production-ready.

5.2. Intelligent Agent Coordination and Modular Processing Pipeline

At the heart of the chatbot's intelligence lies a modular agent-based system, orchestrated primarily by `chatbot.py`, which serves as the central logic router. Each message processed in real time is passed through this module, where it is analyzed and routed to the appropriate component based on intent recognition and query understanding.

The Together AI LLaMA 70B model plays a critical role here. It is used for:

- Recognizing user intent (e.g., ordering, requesting recommendations, small talk).
- Extracting entities like item names, quantities, or preferences.
- Generating coherent, personalized responses in natural language.
- Applying basic safety checks or guard logic to moderate content.

Depending on the intent, the system may involve one or more of its specialized agents:

- The `menu_agent.py` is responsible for handling user queries related to available items. It uses FAISS (Facebook AI Similarity Search) with cosine similarity to implement Retrieval-Augmented Generation (RAG). This allows the system to embed the user's query into a vector and search a database of pre-embedded menu items, returning the most semantically relevant matches in real time.
- The `recommendation_agent.py` builds on the results of the menu agent. It leverages the FP-Growth algorithm, a fast and scalable pattern-mining algorithm, to identify frequently bought-together itemsets. When a user selects or queries an item, this agent suggests complementary items based on historical data patterns, enhancing the ordering experience and increasing user engagement.

The modular design of these agents means that each one operates independently but in coordination, thanks to a shared interface defined by `chatbot.py`. This separation of concerns allows for better maintainability, easy testing, and flexibility to extend the system. For example, introducing a new agent—such as one for handling feedback or FAQs—requires minimal change to the existing structure.

Together, these components form a powerful pipeline where user messages are intelligently processed, relevant data is retrieved or inferred, responses are generated naturally, and actions are taken accordingly. The combination of real-time responsiveness and intelligent, modular logic makes the system capable of delivering a truly interactive and human-like experience.

Chapter 6: Results, Challenges & Future Work

6.1. System Outcomes, Performance Insights, and Development Challenges

The implemented intelligent chat system has achieved its core goal: providing real-time, context-aware, and interactive assistance tailored for small businesses. The system facilitates smooth user interaction via natural language, offering intelligent menu retrieval, personalized recommendations, and seamless ordering workflows. Upon deployment, the chatbot successfully:

- Responds instantly to user inputs via WebSockets.
- Retrieves relevant items using FAISS with cosine similarity.
- Provides contextual recommendations using FP-Growth.
- Maintains conversation continuity through FastAPI's session management.
- Stores session histories reliably in MongoDB.

These features make the system suitable for applications like digital kiosks, customer service automation, and smart ordering platforms for small-to-medium enterprises.

However, the development process came with several technical challenges:

- **Session Context Retention:** Designing a mechanism to retain temporary state in FastAPI while also offloading it to MongoDB post-session required fine-grained control over data flow.
- **Third-Party API Coordination:** Integrating the LLaMA 70B model through Together AI involved overcoming issues like latency, rate limits, and formatting inconsistencies in outputs.
- **Vector Search Optimization:** FAISS had to be fine-tuned with correct thresholds and normalization to avoid irrelevant or mismatched results.
- **Sparse Recommendation Data:** The FP-Growth algorithm performed inconsistently when historical data was limited, requiring synthetic data generation for testing and tuning.
- **Debugging WebSockets:** Since WebSockets operate on a persistent connection, handling disconnects, retries, and error logging demanded specialized attention.

Despite these hurdles, the modular structure of the system (with clearly separated agents and handlers) allowed developers to isolate and resolve issues efficiently.

6.2 Scope for Improvement and Future Enhancements

While the system functions well in its current state, there are multiple opportunities to enhance it further—both in terms of scalability and user experience:

- **Dedicated Intent Classifier:** Currently, the LLaMA model handles both response generation and intent recognition. Offloading intent classification to a lightweight, custom-trained classifier could reduce model load and improve response speed.
- **Advanced Guard Logic:** At present, basic moderation is handled by the language model itself. Implementing a dedicated content moderation pipeline using predefined rules or external moderation APIs would improve safety and reliability in user-facing environments.
- **User Profiling & Personalization:** By leveraging stored chat histories in MongoDB, future versions can introduce user profiles that remember past orders, preferences, and behaviors. This would allow the chatbot to make even more personalized recommendations and re-engage returning users.
- **Scalability Enhancements:** To support higher user traffic, the system can be extended with asynchronous message queues (like RabbitMQ or Kafka) and distributed processing across multiple agents or containers.
- **Voice and Mobile Integration:** Adding voice interface support (using libraries like Whisper for transcription and TTS for speech) and deploying the frontend as a mobile application would dramatically increase accessibility and usage scenarios.
- **Analytics and Admin Dashboard:** An administrative interface could allow business owners to visualize chat statistics, user behavior, peak interaction times, and recommendation performance—turning the chatbot into a tool for both service and strategy.

In essence, the current implementation lays a solid foundation, and with incremental upgrades, the system can evolve into a robust AI-powered assistant capable of serving diverse business needs.

Conclusion

Conclusion

The implementation of the LLaMA-based intelligent chatbot for coffee shops has successfully fulfilled its primary objective: providing a seamless, interactive, and context-aware customer experience. By combining real-time natural language processing, personalized recommendations, and efficient session management, the system has demonstrated its capability to assist customers in an engaging and intuitive manner.

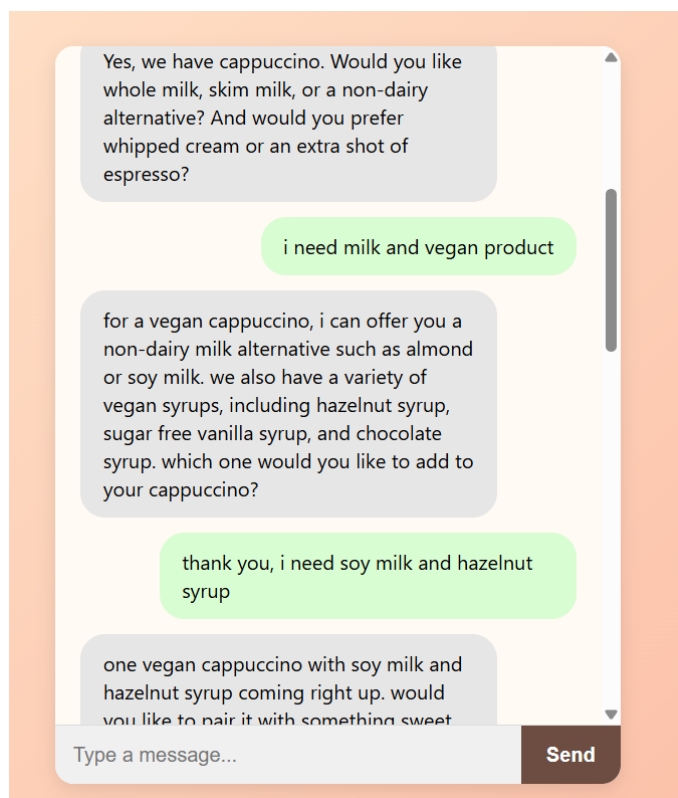
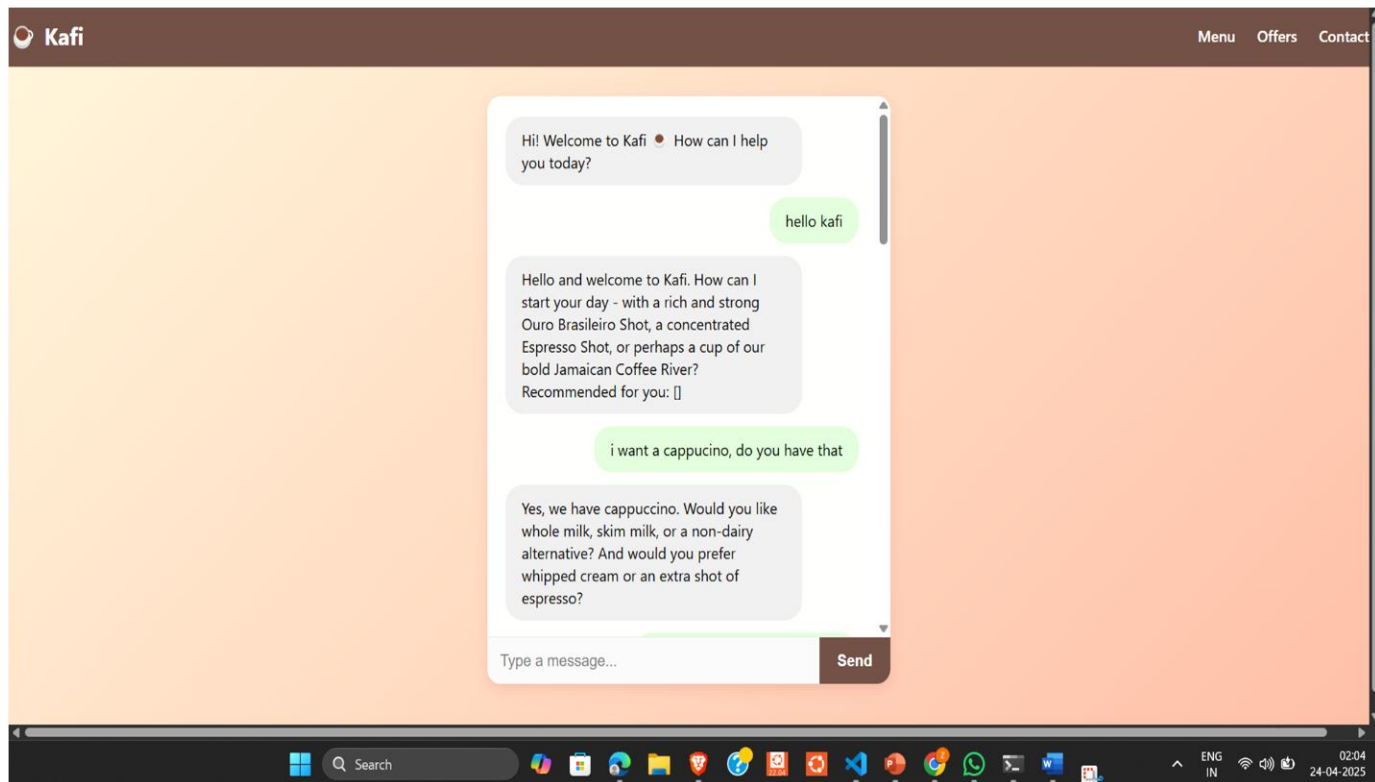
Key features, including instant responses through WebSockets, menu retrieval via FAISS, and contextual recommendations powered by FP-Growth, allow the chatbot to effectively address common customer needs in a coffee shop setting. The integration of MongoDB for session management and data storage ensures that each interaction is preserved, allowing for further personalization and analytics.

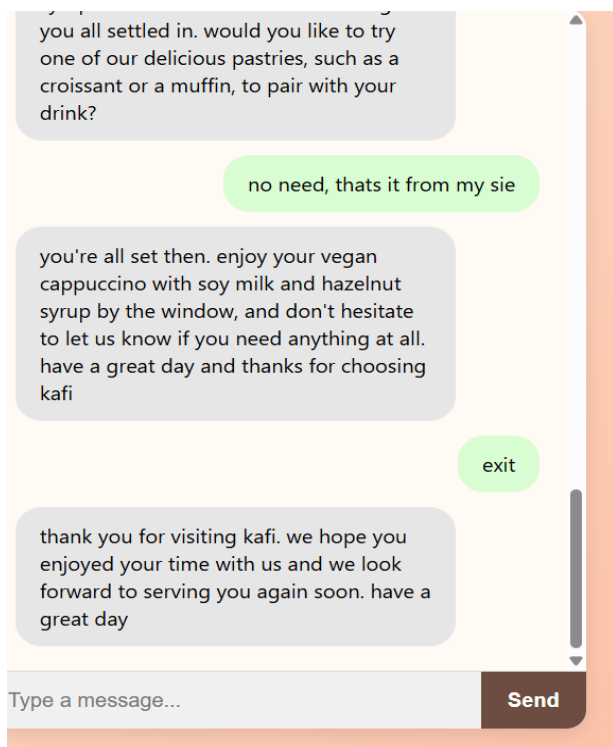
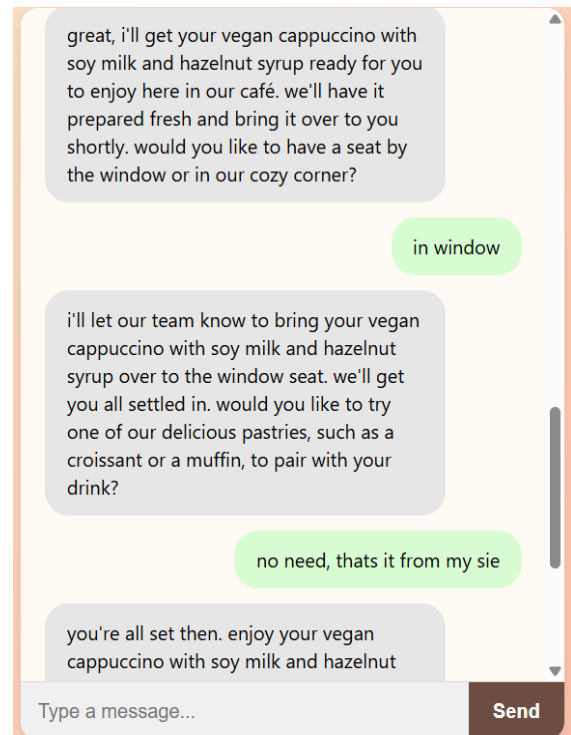
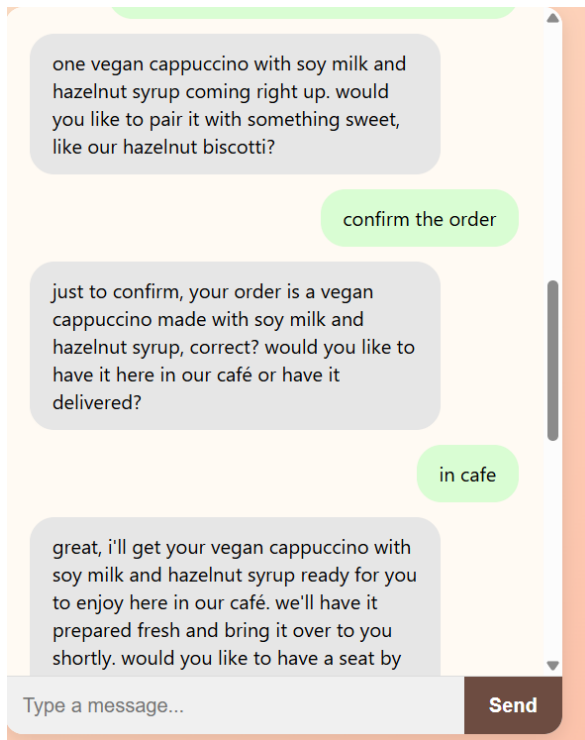
Throughout the development process, various challenges were encountered, such as ensuring session context retention, optimizing the FAISS search for accuracy, and coordinating with third-party APIs like Together AI for the LLaMA model. Despite these hurdles, the system's modular architecture enabled efficient problem-solving and ensured that the project could move forward with scalability in mind.

Looking ahead, the project offers numerous avenues for future enhancements, including the introduction of a dedicated intent classifier, advanced moderation logic, and improved scalability mechanisms. Additional features like voice integration, mobile support, and an analytics dashboard could further elevate the user experience and broaden the system's applicability for businesses beyond the coffee shop domain.

In conclusion, this project not only lays the groundwork for future AI-driven customer interaction solutions but also demonstrates the potential of intelligent chat systems to streamline operations, enhance user engagement, and drive business growth in small to medium enterprises.

Outputs





References

References

1) FAISS (Facebook AI Similarity Search):

The Faiss Library

Authors: Matthijs Douze, Maximilien Guzhva, Vincent Leordeanu, Jeff Johnson, Hervé Jégou, and Tomasz Malisiewicz

Published: January 2024

2) Retrieval-Augmented Generation (RAG):

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Authors: Patrick Lewis, Ethan Perez, Aleksandara Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela

Published: May 2020